

Data Structure and Algorithm

COURSE CODE: CS-201

TOPIC: DOUBLY LINK LIST



Goals for today

To study:

- Linkedlist
 - Doubly/ Two-way linked list

CLO Covered

CLO1: **Explain the** fundamentals of data structure as well as algorithms that operate on them

CLO2: **Apply** and design various data structures methods for specified applications.

Two-Way List

What we have discussed till now is a **one-way** list [Only one way we can traversed the list]

- In a single linked list, every node has a link to its next node in the sequence.
- So, we can traverse from one node to another node only in one direction and we can not traverse back.
- We can solve this kind of problem by using a double linked list.

Two-way List : Can be traversed in two direction

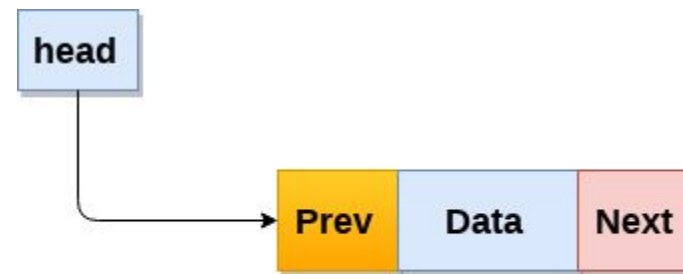
- Forward : From beginning of the list to end
- Backward: From end to beginning of the list

Doubly Linked List

Double linked list is a sequence of elements in which every element has links to its previous element and next element in the sequence.

So, we can traverse forward by using the next field and can traverse backward by using the previous field.

Every node in a double linked list contains three fields and they are shown in the following figure...



Node

Two-Way List

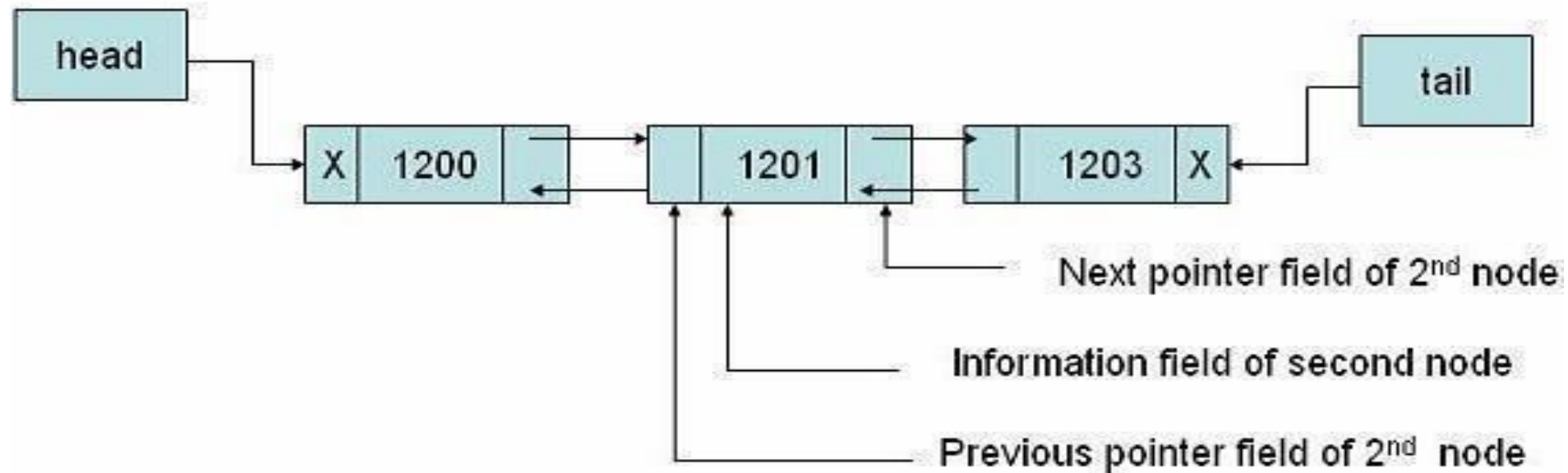
A **two-way list** is a linear collection of data element called nodes where each node **N** is divided into **three parts**:

- An **information field** INFO which contains the data of N
- A **pointer field** 'NEXT_link' which contains the location of the next node in the list
- A **pointer field** 'PREVIOUS_link' which contains the location of the preceding node in the list

Two-Way List

List requires two pointer variables:

- **HEAD**: which points to the first node in the list
- **TAIL**: which points to the last node in the list



Difference

single linked list

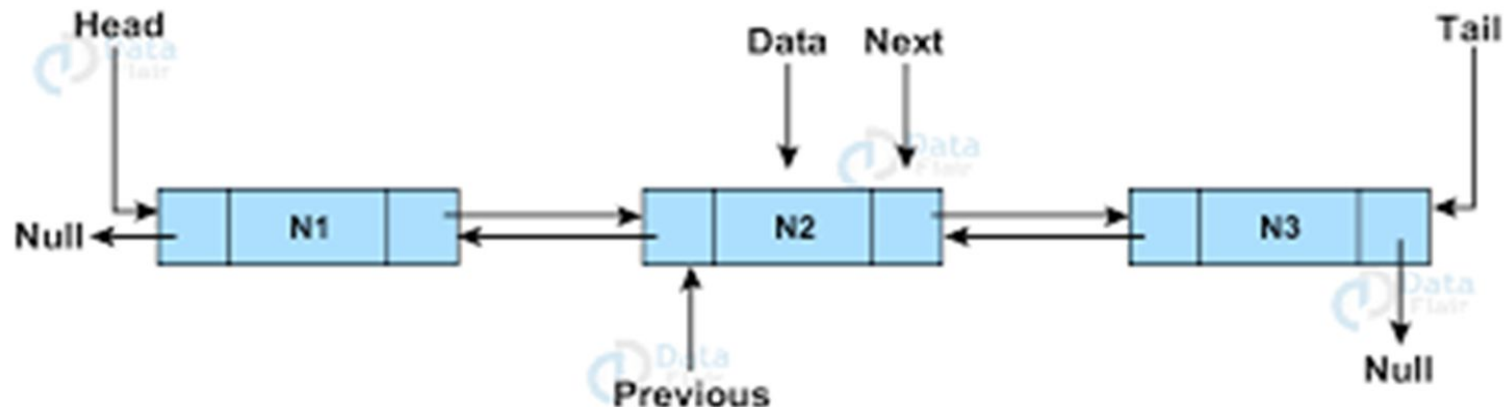
- every node has a link to its next node in the sequence.
- we can traverse from one node to another node only in one direction we can not traverse back

double linked list

- every node has a link to its previous node and next node.
- So, we can traverse forward by using the next field and can traverse backward by using the previous field

Important Points to be remember

- In double linked list, the first node must be always pointed by head.
- Always the **previous field** of the **first node must be NULL**.
- Always the **next field** of the **last node must be NULL**.



Applications/Uses of doubly linked list in real life

- Doubly linked list can be **used in navigation systems** where both front and back navigation is required.
- It is **used by browsers** to implement backward and forward navigation of visited web pages i.e. back and forward button.
- It is also used by various application **to implement Undo and Redo** functionality.
- It can also be **used to represent deck of cards in games**. It is also used to represent various states of a game.

Doubly Linked List Declaration

```
struct node
{
int data;
struct node *prev;
struct node *next;
}*head, *tail;
```

Operation in two-way list

Traversing

Searching

Deletion

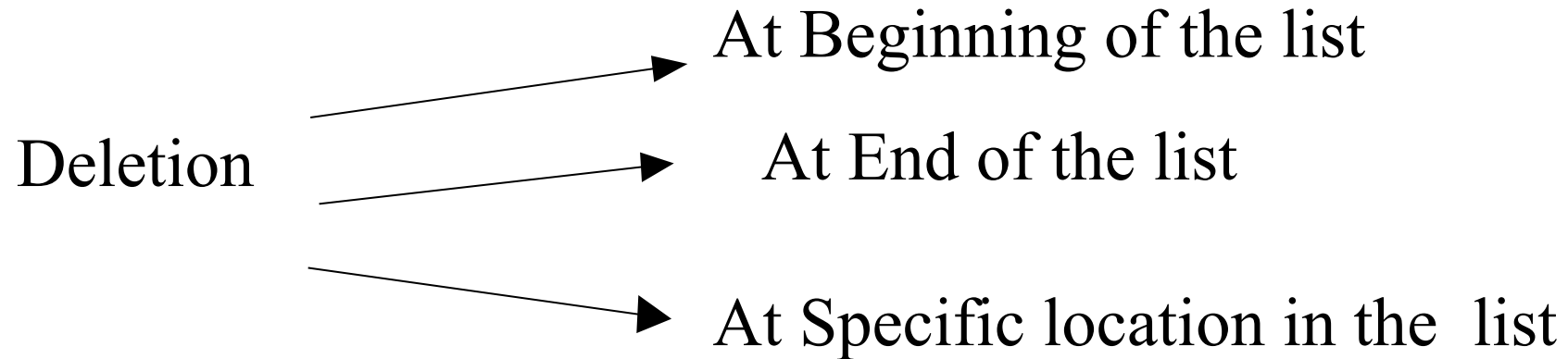
Insertion

Display

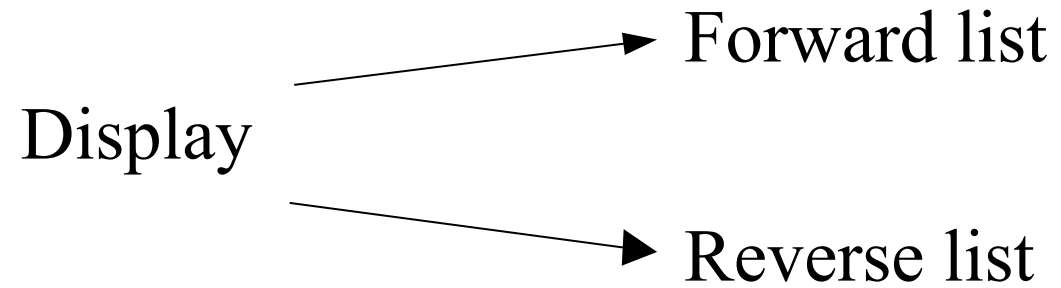
Operation in two-way list

- Insertion
 - At Beginning of the list
 - At End of the list
 - At Specific location in the list

Operation in two-way list



Operation in two-way list



Traversing in two-way list

struct node *p

if (start == Null), then print underflow & return.

p=start; **//forward traversing**

```
while(p!=null)
{
    print p->info;
    p= p ->next;
}
```

p=last; **//backward traversing**

```
while(p!=null)
{
    print p->info;
    p= p ->prev;
}
```

Forward Search in Two-way List

Step1: If *Start* = *Null* Then

Print: "Linked List is Empty"

Exit

[End If]

Step 2: Set *Ptr* = *Start* // forward searching

Step 3: Repeat While *Ptr* ≠ *Null*

If *Ptr* → *info* = *Data* Then

Print : "Element is found at address":*Ptr* Exit

Else

Set *Ptr* = *Ptr* → *Next*

[End If]

[End Loop]

Step 4: Print : "Element not found in linked list" Step 5 :Exit

Backward Searching in Two-way List

Step1: If ***Start*** = ***Null*** Then
 Print: "Linked List is Empty"
 Exit
 [End If]

Step 2: Set ***Ptr*** = ***last*** // *backward searching*

Step 3: Repeat While ***Ptr*** ≠ ***Null***
 If ***Ptr*** → ***info*** = ***Data*** Then
 Print : "Element is found at address":***Ptr*** Exit
 Else
 Set ***Ptr*** = ***Ptr*** → ***Prev***
 [End If]
 [End Loop]

Step 4: Print : "Element not found in linked list" Step 5 :Exit

Algorithm for insertion in two-way list- At Start when start is null

if (start == Null)

```
struct node *tmp;  
tmp = (struct node *) malloc(sizeof(struct node))  
tmp → info = 2  
tmp → prev = NULL  
tmp → next = NULL  
start = tmp
```

Algorithm for insertion in two-way list

At Start when start is not null

```
struct node *tmp;  
tmp = (struct node *) malloc(sizeof(struct node))  
tmp → info = 2  
tmp → prev = NULL  
tmp → next = start  
start → prev = tmp  
start = tmp
```

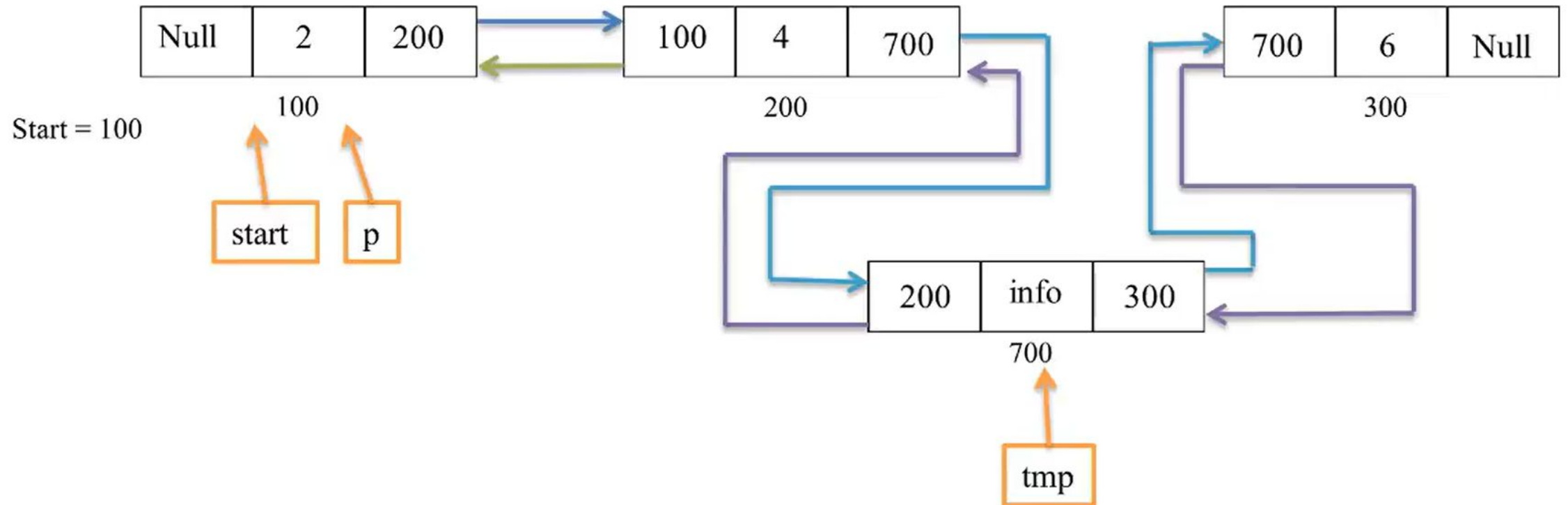
Algorithm for insertion in two way list

At End

```
struct node *tmp, *p;  
tmp = (struct node *) malloc(sizeof(struct node))  
tmp → info = 6  
p = start  
while(p → next != Null)  
    p = p → next  
p → next = tmp  
tmp → next = Null  
tmp → prev = p
```

Algorithm for insertion in two-way list

At any Position



Algorithm for insertion in two-way list

At any Position

```
struct node *temp, *p;
temp=(struct node*)malloc(sizeof(struct node*));
temp->info = data;
if(pos==1) // Insertion at start
{
    temp->prev = NULL;
    temp->next = start;
    start->prev = temp;
    start = temp;
}
```

Algorithm for insertion in two-way list

At any Position

```
p=start; //Finding node insertion position
for(int i = 1; i<pos-1 && p != NULL ; i++)
{
    p=p->link;
}
if(p==NULL) //Checks list is empty
{
    cout<<"List elements are less than specified limit";
}
else // node insertion
temp->prev = p;
temp->next = p->next;
if(p->next!=NULL)
    p->next->prev=temp;
p->next = temp;
```

Questions?

